

August 2025

## Robust Real-Time UAV Target Tracking with Onboard Vision-Based Yaw Control

Rylan Malarchick

*Embry-Riddle Aeronautical University, malarchr@my.erau.edu*

Jose Castelblanco

*Embry-Riddle Aeronautical University, castej10@my.erau.edu*

Enrique Amaya

*Embry-Riddle Aeronautical University, amayavie@my.erau.edu*

Carmen DiMario

*Embry-Riddle Aeronautical University, dimarioc@my.erau.edu*

Graysen Brinkman

*Embry-Riddle Aeronautical University, brinkmag@my.erau.edu*

*See next page for additional authors*

Follow this and additional works at: <https://commons.erau.edu/beyond>



Part of the [Aeronautical Vehicles Commons](#), [Artificial Intelligence and Robotics Commons](#), [Controls and Control Theory Commons](#), [Engineering Physics Commons](#), [Navigation, Guidance, Control and Dynamics Commons](#), [Robotics Commons](#), and the [Software Engineering Commons](#)

---

### Recommended Citation

Malarchick, Rylan; Castelblanco, Jose; Amaya, Enrique; DiMario, Carmen; Brinkman, Graysen; Kumar, Chirag; Yoon, Kiwon; and Berhane, Sajid (2025) "Robust Real-Time UAV Target Tracking with Onboard Vision-Based Yaw Control," *Beyond: Undergraduate Research Journal*: Vol. 9 , Article 6.

Available at: <https://commons.erau.edu/beyond/vol9/iss1/6>

This Article is brought to you for free and open access by the Journals at Scholarly Commons. It has been accepted for inclusion in Beyond: Undergraduate Research Journal by an authorized administrator of Scholarly Commons. For more information, please contact [commons@erau.edu](mailto:commons@erau.edu).

---

# Robust Real-Time UAV Target Tracking with Onboard Vision-Based Yaw Control

## Cover Page Footnote

This work was supported by the Spark grant from the Office of Undergraduate Research at Embry-Riddle Aeronautical University. The authors thank Dr. Sergey Drakunov for providing laboratory space in the Engineering Physics and Propulsion Laboratory (EPPL) and for faculty mentorship throughout this project. The authors also acknowledge the AIRBUD project team for the inherited hardware platform that serves as the foundation for the AIRHOUND system.

## Authors

Rylan Malarchick, Jose Castelblanco, Enrique Amaya, Carmen DiMario, Graysen Brinkman, Chirag Kumar, Kiwon Yoon, and Sajid Berhane



# AIRHOUND: Robust Real-Time UAV Target Tracking with Onboard Vision-Based Yaw Control

Rylan Malarchick<sup>1</sup>, Jose Castelblanco<sup>1</sup>, Enrique Amaya<sup>1</sup>, Carmen DiMario<sup>1</sup>, Graysen Brinkman<sup>1</sup>, Chirag Kumar<sup>1</sup>, Kiwon Yoon<sup>1</sup>, and Sajid Berhane<sup>1</sup>

<sup>1</sup>Embry-Riddle Aeronautical University, Daytona Beach, FL, USA

June 15, 2026

---

## Abstract

The proliferation of small, agile unmanned aerial vehicles (UAVs) has created a growing need for platforms that can autonomously detect and maintain visual contact with moving aerial targets. Achieving this on resource-constrained airborne hardware is difficult: modern detection networks are computationally demanding, and purely reactive controllers fail during momentary detection dropouts. This study investigates whether a modular ROS2-based software architecture can achieve reliable real-time yaw tracking using onboard computer vision, validated for sustained operation. We present AIRHOUND (Autonomous Intelligent Rotorcraft for Hostile Object Unified Navigation and Detection), a UAV platform implementing vision-based “yaw-to-target” control. AIRHOUND employs a three-node ROS2 architecture (perception, tracking, offboard control). YOLOv8 object detection is optimized with NVIDIA TensorRT for an embedded NVIDIA Jetson Orin; a geometric tracking module converts pixel coordinates to angular yaw errors using a pinhole camera model, and a proportional controller streams rate-limited yaw commands to a PX4 flight controller through the Micro XRCE-DDS bridge. The architecture was validated with a 600-second end-to-end integration test in the PX4 Software-In-The-Loop (SITL) simulation environment using a 30 Hz detector. The pipeline sustained 30 Hz message throughput across all stages with zero dropped messages, exceeding the 15 Hz minimum requirement by a factor of two and maintaining a 0.000 Hz steady-state standard deviation, and seven integration bugs were identified and resolved before validation succeeded. This work contributes a validated, reusable software foundation for vision-based UAV tracking whose modular structure decouples perception, tracking, and control. Future work moves from simulation to physical validation on the assembled hardware, transitions to a transformer-based detector (RF-DETR), and adds Kalman filtering for occlusion-robust tracking.

**Keywords:** UAV target tracking; YOLOv8; TensorRT; ROS2; PX4 offboard control; embedded computer vision

---

## 1 Background

The proliferation of small, agile Unmanned Aerial Vehicles (UAVs) has created significant challenges for autonomous detection and tracking systems. Applications ranging from airspace security to search-and-rescue operations increasingly require platforms capable of maintaining persistent visual contact with moving targets. However, achieving robust real-time tracking remains difficult, particularly in cluttered environments where targets may be partially occluded or where sensor data is corrupted by envi-

ronmental conditions (1).

Standard machine learning approaches for object detection, while achieving impressive accuracy on benchmark datasets, often struggle when deployed on resource-constrained embedded hardware. The computational demands of modern neural networks conflict with the size, weight, and power limitations of UAV platforms. Furthermore, purely reactive control systems—those that respond only to current sensor measurements—frequently fail when faced with momentary detection dropouts caused by occlusion, motion blur, or erratic target movement

(2).

This work presents AIRHOUND (Autonomous Intelligent Rotorcraft for Hostile Object Unified Navigation and Detection), a UAV platform designed to address these challenges through a robust, low-latency perception and control pipeline. The system implements a “yaw-to-target” capability wherein the UAV autonomously adjusts its heading to maintain visual lock on a target UAV. The core research question driving this work is: *Can a modular ROS2-based software architecture achieve reliable real-time yaw tracking using onboard computer vision, validated for continuous operation on embedded compute hardware?*

The AIRHOUND platform builds upon hardware inherited from the AIRBUD project, integrating a new flight controller, companion computer, and complete software stack. The perception pipeline employs YOLOv8 object detection optimized with NVIDIA TensorRT for efficient inference on an embedded Jetson computer. Detection outputs are processed through a geometric tracking module that converts pixel coordinates to angular errors using camera intrinsics, then generates yaw commands streamed to a PX4 flight controller via the ROS2 middleware framework (3).

The primary contribution of this work is the validation of a complete end-to-end software architecture through extensive simulation testing. A 600-second continuous integration test demonstrated sustained 30 Hz message throughput with zero dropped messages, confirming the reliability of inter-module communication necessary for real-world deployment. This establishes a validated baseline for future hardware integration and provides a foundation for more advanced predictive control systems.

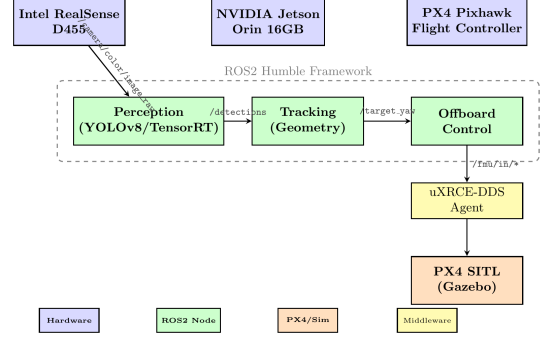
## 2 Methods

### 2.1 System Overview

The AIRHOUND system employs a three-node modular architecture implemented within the ROS2 Humble framework on Ubuntu 22.04. This design enables parallel development of perception, tracking, and control components while maintaining strict interface contracts between modules. Communication with the PX4 flight controller is achieved through the Micro XRCE-DDS bridge, which translates between ROS2 topics and PX4’s internal messaging system (4).

The modular architecture follows a unidirectional data flow: camera images are processed by the perception node to generate object detections, which are consumed by the tracking node to compute yaw errors, which are then converted by the offboard control node into flight controller commands. This separation of concerns allows each module to be developed, tested, and optimized independently while ensuring predictable system behavior. Figure 1 il-

lustrates the complete system architecture and data flow.



**Figure 1: AIRHOUND system architecture showing the three-node ROS2 pipeline. Camera images flow through perception (YOLOv8/TensorRT), tracking (geometric yaw calculation), and offboard control (PX4 command generation) modules. The uXRCE-DDS bridge provides communication with the PX4 flight controller.**

### 2.2 Hardware Platform

The AIRHOUND hardware platform inherits its airframe and basic avionics from the AIRBUD project, with significant upgrades to the sensing and compute subsystems. Table I summarizes the key hardware specifications.

**Table I: AIRHOUND Hardware Specifications**

| Component          | Specification            | Details                                            |
|--------------------|--------------------------|----------------------------------------------------|
| Vision Sensor      | Intel RealSense D455     | 640×480 to 1280×720<br>30 fps, 87°×58° FOV         |
| Companion Computer | NVIDIA Jetson Orin       | 16 GB unified memory<br>TensorRT inference support |
| Flight Controller  | PX4 v1.14+               | Pixhawk-compatible<br>6-axis IMU                   |
| Software Framework | ROS2 Humble<br>uXRCE-DDS | Ubuntu 22.04 LTS<br>PX4-ROS2 bridge                |

The Intel RealSense D455 depth camera provides the primary visual input, selected for its wide field of view and accurate depth sensing at extended ranges. While depth data is available for future range estimation, the current implementation uses only the RGB color stream for object detection. The NVIDIA Jetson Orin serves as the companion computer, providing sufficient computational resources for real-time neural network inference while meeting the power constraints of an airborne platform.

### 2.3 Perception Module

The perception module detects target UAVs in the camera image stream and publishes structured detection messages for downstream processing. The implementation employs YOLOv8, a state-of-the-art convolutional neural network architecture for real-time object detection (2).

For deployment on the embedded Jetson platform, the YOLOv8 model is optimized using NVIDIA TensorRT, which performs layer fusion, precision calibration, and kernel auto-tuning to maximize inference throughput. The model is exported to FP16 (half-precision floating point) format, which provides significant speedup on Tensor Core-equipped GPUs while maintaining detection accuracy within acceptable bounds.

The perception node subscribes to the camera color image topic and camera intrinsics from the RealSense ROS2 driver. For each input frame, the node performs inference and publishes detections as `vision_msgs/Detection2DArray` messages. Each detection includes bounding box center coordinates  $(u, v)$  in pixels, bounding box dimensions, an object class identifier, and a detection confidence score.

The target performance specification requires detections at  $\geq 15$  Hz with median latency  $\leq 60$  ms at 720p resolution. For simulation testing, a mock detector node publishes synthetic detections at 30 Hz, enabling end-to-end pipeline validation without requiring actual camera hardware or neural network inference.

## 2.4 Tracking and Geometry Module

The tracking module converts 2D pixel coordinates from detections into angular yaw errors suitable for flight controller consumption. This transformation employs the pinhole camera model, which relates pixel coordinates to angular displacements using the camera's intrinsic parameters (5). Figure 2 illustrates the standard pinhole camera geometry showing how 3D world points project onto the 2D image plane.

The tracking module must account for differing coordinate conventions between the camera and vehicle body frames. The camera optical frame follows optical conventions (+Z forward along the optical axis, +X right, +Y down), whereas the UAV body frame follows the aerospace North-East-Down (NED) convention (+X forward, +Y right, +Z down). A point expressed in the camera frame,  $\mathbf{p}^c = [x_c, y_c, z_c]^\top$ , maps to the body frame through a fixed rotation,

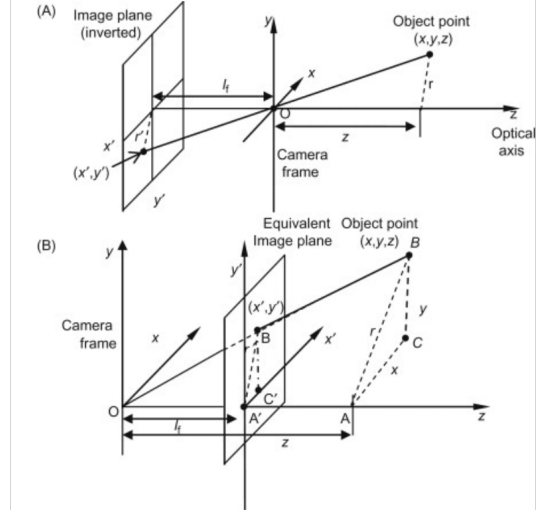
$$\mathbf{p}^b = R_c^b \mathbf{p}^c, \quad (1)$$

where, for a camera rigidly mounted with its optical axis aligned to the vehicle forward direction,

$$R_c^b = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}. \quad (2)$$

Under this mounting assumption the camera's optical axis coincides with the body +X axis, so a purely horizontal image displacement produces a rotation about the body +Z (yaw) axis with no roll or pitch coupling. This reduces the full three-dimensional transformation of eq. (1) to the scalar

horizontal-error treatment used below; future work replaces the fixed  $R_c^b$  with runtime ROS2 TF2 transforms to accommodate arbitrary mounting orientations.

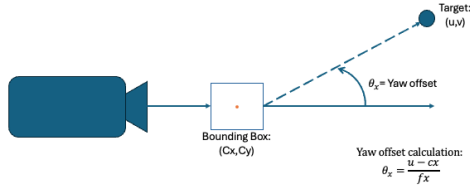


**Figure 2: Pinhole camera model showing projection of 3D world coordinates  $(x, y, z)$  onto the 2D image plane at  $(x', y')$ . (A) Traditional model with inverted image plane behind the optical center. (B) Equivalent model with virtual image plane in front, simplifying the projection geometry. The focal length  $l_f$  determines the relationship between angular displacement and pixel offset.**

Given a detection with bounding box center at pixel coordinates  $(u, v)$  and camera intrinsics  $(f_x, f_y, c_x, c_y)$  obtained from the `CameraInfo` message, the horizontal angular error (yaw) is computed as:

$$e_x = \frac{u - c_x}{f_x} \approx \tan^{-1} \left( \frac{u - c_x}{f_x} \right). \quad (3)$$

This relationship derives from the pinhole camera model, where the small-angle approximation  $\theta \approx \tan(\theta)$  holds for targets near the optical center. The approximation remains accurate within 1% for angles up to approximately  $\pm 10^\circ$ , which covers the central region of the D455's  $87^\circ$  horizontal field of view where active tracking maintains the target. Figure 3 illustrates this geometric relationship between pixel displacement and yaw angle.



**Figure 3: Geometric relationship for yaw error calculation.** The camera’s optical center  $(c_x, c_y)$  defines the zero-error reference point. When a target is detected at pixel coordinates  $(u, v)$ , the horizontal offset  $(u - c_x)$  divided by the focal length  $f_x$  yields the yaw correction angle  $\theta_x$  in radians.

The tracking node implements a proportional (P) controller that maps yaw error to yaw rate commands. To prevent oscillation and jitter around the setpoint, the controller incorporates rate limiting (output clamped to  $\pm 1.0$  rad/s by default) and a deadband (errors below 0.01 rad produce zero output). When the target is lost, the module enters a predictive mode that extrapolates the target position based on the last observed angular velocity, providing continued yaw commands during brief occlusion events and preventing the controller from reverting to a hold state prematurely. The tracking node publishes yaw rate commands as `std_msgs/Float32` messages at the same rate as incoming detections.

## 2.5 Offboard Control Module

The offboard control module translates yaw commands from the tracking module into PX4-compatible messages for flight controller execution. PX4’s Offboard flight mode enables external computers to command vehicle attitude, position, or velocity setpoints, but imposes strict timing requirements: the flight controller will exit Offboard mode if setpoint messages are not received at  $\geq 2$  Hz (1).

The offboard control node continuously publishes `OffboardControlMode` and `TrajectorySetpoint` messages at 10–20 Hz; after establishing a stable setpoint stream it commands the flight controller to enter Offboard mode and arms the vehicle. Incoming target yaw values are mapped to the yaw field of `TrajectorySetpoint`. A watchdog publishes hold commands (zero yaw rate) if yaw messages are not received within a timeout period, maintaining stable flight. Communication with PX4 occurs through the uXRCE-DDS bridge, which runs a Micro XRCE-DDS Agent on the companion computer connected to the DDS client integrated in the PX4 firmware, providing topic-based communication without custom serial protocols.

## 2.6 Simulation and Testing Environment

End-to-end system validation was performed using the PX4 Software-In-The-Loop (SITL) simulator with Gazebo for physics and visualization (6). This environment provides a complete flight controller implementation with realistic sensor models, enabling validation of the ROS2 software architecture without requiring flight hardware. The integration test launched PX4 SITL with Gazebo, started the Micro XRCE-DDS Agent, and brought up the perception (mock detector at 30 Hz), tracking, and offboard control nodes; all topics were recorded with `rosviz` for 600 seconds and analyzed for message rate and timing. The 600-second (10-minute) duration was selected to demonstrate sustained operation, as real-world tracking missions may require extended continuous operation. Success criteria included maintaining target message rates with zero dropped messages across all pipeline stages.

## 3 Results

### 3.1 End-to-End Integration Test

The 600-second integration test successfully validated the complete three-node pipeline operating in the PX4 SITL environment. Table II summarizes the key performance metrics extracted from `rosviz` analysis.

**Table II: End-to-End Integration Test Results (600-second duration)**

| Metric              | Perception | Tracking | Req.       |
|---------------------|------------|----------|------------|
| Total Messages      | 17,993     | 17,993   | —          |
| Mean Rate (Hz)      | 30.00      | 30.00    | $\geq 15$  |
| Std. Deviation (Hz) | 0.000      | 0.000    | $< 1$      |
| Min Rate (Hz)       | 30         | 30       | $\geq 15$  |
| Max Rate (Hz)       | 30         | 30       | —          |
| Message Drops       | 0          | 0        | 0          |
| Test Duration (s)   | 599.7      | 599.7    | $\geq 300$ |

Both the perception and tracking nodes maintained exactly 30 Hz operation throughout the steady-state test period, with zero variance in per-second message counts. The only deviation occurred during the first second of operation (startup transient), where the perception node published 23 messages and the tracking node published 26 messages. This startup behavior is expected as nodes initialize their publishers and subscribers. Excluding the startup transient, the system achieved 599 of 599 seconds at exactly 30 Hz for perception, 598 of 599 seconds at exactly 30 Hz for tracking, and zero message drops across the entire test duration. Figure 4 presents the performance summary from the integration test.

| AIRHOUND End-to-End Integration Test Results<br>600-Second Continuous Operation |            |          |              |        |
|---------------------------------------------------------------------------------|------------|----------|--------------|--------|
| Metric                                                                          | Perception | Tracking | Requirement  | Status |
| Total Messages                                                                  | 17,993     | 17,966   | —            | —      |
| Mean Rate (Hz)                                                                  | 30.00      | 30.00    | $\geq 15$ Hz | PASS   |
| Std Dev (Hz)                                                                    | 0.000      | 0.000    | $< 1$ Hz     | PASS   |
| Min Rate (Hz)                                                                   | 30         | 30       | $\geq 15$ Hz | PASS   |
| Max Rate (Hz)                                                                   | 30         | 30       | —            | —      |
| Message Drops                                                                   | 0          | 0        | 0            | PASS   |
| Duration                                                                        | 599.7s     | 599.7s   | $\geq 300$ s | PASS   |

**Figure 4: AIRHOUND end-to-end integration test results over 600 seconds of continuous operation. The system maintained exactly 30 Hz message throughput with zero standard deviation and zero message drops, exceeding the 15 Hz minimum requirement by a factor of two.**

### 3.2 Integration Issues and Resolutions

The integration testing process identified seven software bugs that were resolved prior to successful validation. These issues, summarized in Table III, highlight the importance of clean-room testing in a controlled environment matching the target deployment platform. All bugs were identified and fixed in the controlled simulation environment, preventing potential failures during flight testing.

**Table III: Integration Bugs Identified and Resolved**

| Category               | Description                                                                      |
|------------------------|----------------------------------------------------------------------------------|
| Python module naming   | Package directory used a hyphen instead of an underscore, causing import failure |
| Message field access   | Incorrect access pattern for the <code>vision_msgs</code> detection score field  |
| Bounding box access    | Incorrect access pattern for bbox center position coordinates                    |
| Missing source files   | Entry points referenced modules not present in the package                       |
| ROS2 API compatibility | QoS profile function-call syntax incompatible with Humble                        |
| Build configuration    | <code>CMakeLists.txt</code> missing entries for executable nodes                 |

## 4 Discussion

### 4.1 Interpretation of Results

The 600-second integration test demonstrates that the AIRHOUND software architecture achieves the reliability necessary for real-world deployment. The sustained 30 Hz operation with zero message drops indicates robust inter-node communication through

the ROS2 middleware. The perfect steady-state performance (30.00 Hz mean, 0.000 Hz standard deviation) validates the timing characteristics of the ROS2 publish-subscribe model for this application, and the system exceeds the minimum 15 Hz requirement by a factor of two, providing margin for future algorithmic complexity or additional processing stages.

### 4.2 Significance

This work establishes a validated software foundation for vision-based UAV tracking. The demonstrated reliability over extended operation periods addresses a key concern for autonomous systems: ensuring consistent behavior during prolonged missions. The modular three-node architecture successfully decoupled perception, tracking, and control development, enabling parallel work streams while maintaining strict interface contracts, and provides a template for similar perception-to-control pipelines in other robotic applications. While the current validation used mock detections, the architecture is designed to accommodate real neural network inference with minimal modification.

### 4.3 Limitations

Several limitations constrain the interpretation of these results. First, all testing was performed in SITL simulation; while the hardware platform is fully assembled, the tracking algorithms have not yet been validated on physical hardware. Second, the perception module used synthetic detections at a fixed rate rather than actual YOLOv8 inference, which introduces variable latency and potential detection failures not captured in this test. Third, the test validated software connectivity but did not compare tracking performance against alternative approaches, nor did it measure actual yaw tracking accuracy (error between commanded and achieved heading). Finally, the current implementation handles only single-target scenarios; multi-target tracking and target-selection logic remain for future work.

### 4.4 Future Work: Toward Physical Validation

The immediate next phase moves the validated software stack from simulation onto the assembled AIRHOUND airframe. Physical testing will proceed in stages: (1) bench tests streaming live TensorRT-optimized YOLOv8 inference from the Jetson Orin and RealSense D455 to confirm true end-to-end latency from image capture to yaw command; (2) tethered hover tests in which the vehicle is mechanically restrained while it yaws to track a moving target, allowing the proportional controller and watchdog logic to be exercised safely; and (3) free-flight tracking trials following established pre-flight checklists and safety protocols developed by the flight operations team. These trials will record yaw track-



ing accuracy (commanded versus achieved heading), detection-to-command latency, and occlusion-recovery behavior—metrics the simulation test did not capture.

Beyond hardware bring-up, the platform will transition from convolutional detection to transformer-based detection. This follow-on research, “Predictive Target Pursuit for Autonomous UAVs Using a Real-Time Fast Detection Transformer (RF-DETR),” has been accepted for poster presentation at the SPIE Defense + Security 2026 conference (Machine Learning from Challenging Data, Paper 14030-26, National Harbor, MD). RF-DETR offers promise for improved performance in cluttered and occlusion-prone environments through its relation-aware attention mechanisms, with the current YOLOv8 implementation serving as the comparative baseline. Planned tracking-module upgrades include runtime TF2 coordinate transforms (replacing the fixed mounting assumption of eq. (2)), a proportional-derivative (PD) controller for smoother transient response, and a Kalman filter to replace linear extrapolation during target dropout with uncertainty-aware, confidence-weighted state estimation.

## 5 Conclusion

The AIRHOUND project successfully developed and validated a modular ROS2-based software architecture for real-time UAV target tracking. The 600-second end-to-end integration test demonstrated sustained 30 Hz operation with zero message drops, confirming the reliability of the perception-tracking-control pipeline. While current validation is limited to simulation, the established software foundation and assembled hardware platform position the project for the staged hardware integration and flight testing described above.

### *Acknowledgments*

This work was supported by the Spark grant from the Office of Undergraduate Research at Embry-Riddle Aeronautical University. The authors thank Dr. Sergey Drakunov for providing laboratory space in the Engineering Physics and Propulsion Laboratory (EPPL) and for faculty mentorship throughout this project, and acknowledge the AIRBUD project team for the inherited hardware platform that serves as the foundation for the AIRHOUND system.

### *Disclosure statement:*

No potential conflict of interest is reported by the authors.



## References

- [1] PX4 Development Team. “PX4 Autopilot User Guide”, 2024. **available at:** <https://docs.px4.io/main/en/> (accessed 10 Dec. 2024).
- [2] Jocher, Glenn, Chaurasia, Ayush, and Qiu, Jing. “Ultralytics YOLOv8”, 2023. **available at:** <https://github.com/ultralytics/ultralytics> (accessed 10 Dec. 2024).
- [3] Open Robotics. “ROS 2 Humble Hawksbill Documentation”, 2024. **available at:** <https://docs.ros.org/en/humble/> (accessed 10 Dec. 2024).
- [4] eProsima. “Micro XRCE-DDS: An XRCE implementation for embedded systems”, 2024. **available at:** <https://micro-xrce-dds.docs.eprosima.com/en/latest/> (accessed 10 Dec. 2024).
- [5] ROS-Perception. “vision\_msgs: ROS messages for computer vision”, 2024. **available at:** [https://github.com/ros-perception/vision\\_msgs](https://github.com/ros-perception/vision_msgs) (accessed 10 Dec. 2024).
- [6] Koenig, Nathan and Howard, Andrew. “Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator”, *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2004, pp. 2149–2154. DOI: 10.1109/IROS.2004.1389727.